SOFTWARE TESTING · MACHINE LEARNING

Fuzz Testing for Machine Learning

Coverage-guided fuzzing of ResNet-50, ViT-Base and the DistilBERT tokenizer

⚠ **AI-generated content (Claude):** code, experiments, text and slides. Review before use or presentation.

October 28, 2026 · leonardorh18/ml-fuzzy-tests

WHAT IS FUZZ TESTING

Feeding a system generated inputs until something breaks

Instead of writing test cases by hand, a program (the **fuzzer**) generates thousands of inputs automatically and observes how the system behaves.

> Classic example
>
> A PDF reader receives thousands of slightly corrupted files. One of them crashes the program: a bug found before a user, or an attacker, finds it.

1. Generate an input
at random, or by mutating an existing one

↓

2. Run the system
and record what it exercised

↓

3. Check: did it fail?
the rule that decides is the **oracle**

↓

4. Keep it if it explored something new
guided by **coverage**; repeat thousands of times

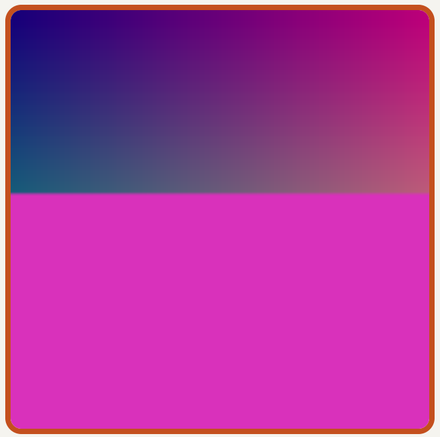
FUZZ TESTING FOR MACHINE LEARNING

In ML, a failure is the model changing its mind



Original

ViT: class 614



Mutated (brightness)

ViT: class 727



Difference ×6

≈ 0.08 per pixel

What changes from classical fuzzing

Input

slightly perturbed images and text

Oracle

models rarely crash; a failure is an unjustified change in prediction

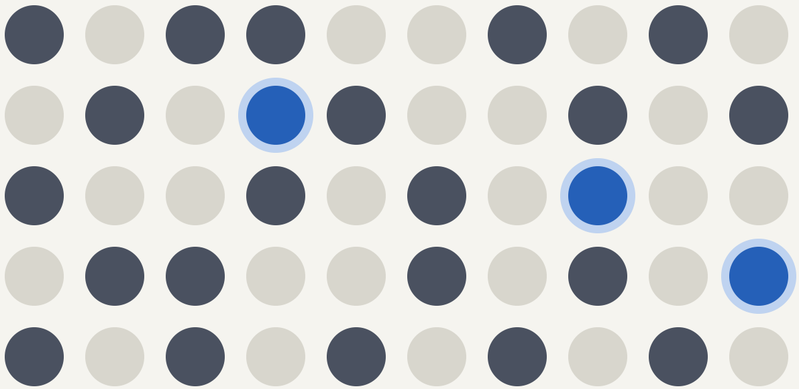
Coverage

activated neurons, not lines of code

Real example from our run (iteration 20) · classes are numbered because the ViT ran with random weights · AI-generated content · 3

HOW IT WORKS · NEURON COVERAGE

The fuzzer keeps images that activate new neurons



never activated already covered **new in this mutant**

Illustrative sketch of one layer

Neuron

one channel of a ResNet block or one hidden dimension of a ViT block

Covered

some input has activated it above 50% of the layer's maximum

Feedback

a non-failing mutant that activates new neurons returns to the corpus and spawns new mutants

Idea from DeepXplore (2017) and TensorFuzz (2019)

MODELS

Two image classifiers and one text tokenizer

ResNet-50

Purpose: identify the object in a photo among the 1,000 ImageNet classes.

How: a 50-layer convolutional network whose filters detect edges, textures and shapes.

```
photo of a dog
→ "golden retriever"
```

ViT-Base

Purpose: the same task as ResNet-50, with the same 1,000 classes.

How: splits the photo into 16×16-pixel patches and applies attention, the technique behind language models.

```
photo of a mug
→ "coffee mug"
```

Tokenizer

Purpose: split text into pieces (tokens) and map each one to a number that DistilBERT understands.

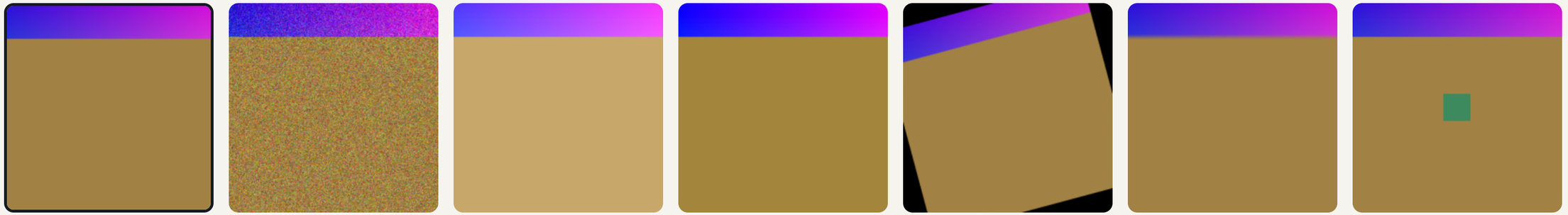
Why test it: every text passes through it before reaching the model.

```
"the cat is sleeping"
→ [CLS] the cat is sleeping
[SEP]
```

ResNet and ViT perform the same task with different architectures, so both can be compared on identical images.

TEST GENERATION · IMAGES

Six mutations, clipped to at most 0.08 per pixel



original noise brightness contrast rotation blur patch

1. Sample an image from the corpus

2. Apply a random mutation

3. Clip each pixel to ±0.08 of the original

4. Classify and measure neurons

5. Class changed? **Failure**: save the image

Mutations at the top of each range, before clipping to 0.08 · synthetic seed used in the run · AI-generated content · 6

TEST GENERATION · TEXT

Atheris generates bytes; four oracles judge the tokenizer

generated bytes
`0a 91 01 00 00 00`

→

decoded input
`max_length=2, 'Ƒ'`

→

tokenizer output
`[CLS] x [SEP]`

1. No exception
the call must never fail

2. Length limit
tokens ≤ max_length; here, 3 > 2

3. Valid ids
every id exists in the vocabulary

4. Stable round trip
decoding and re-encoding changes nothing further

Atheris keeps the inputs that execute new lines of the tokenizer's Python code and mutates them further: classical fuzzing, guided by code coverage.

EXPERIMENTAL SETUP

Identical conditions for both models

Parameter	Value
Iterations	500 per model
Seeds	6 synthetic images
Maximum perturbation (ε)	0.08 per pixel
Coverage threshold	0.5
CPU time	17 s ResNet · 43 s ViT
Tokenizer	61 s fast · < 1 s Python

⚠

Random weights

The environment could not reach the Hugging Face Hub. The models ran with their published architectures and **random weights**, and the tokenizer with a local vocabulary.

The experiment validates the method; the numbers do not measure the robustness of the trained models.

RESULTS · IMAGES

ViT: 45 failures in 500. ResNet: none.

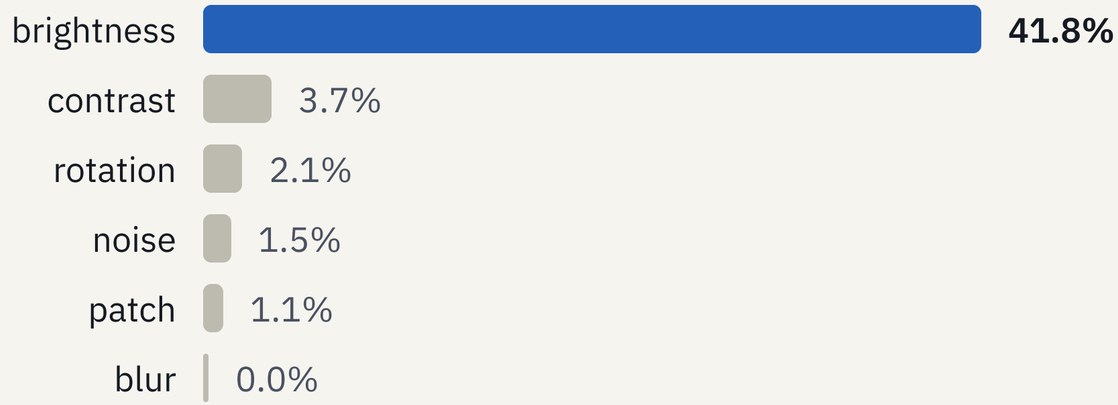
ViT-Base

45 / 500

ResNet-50

0 / 500

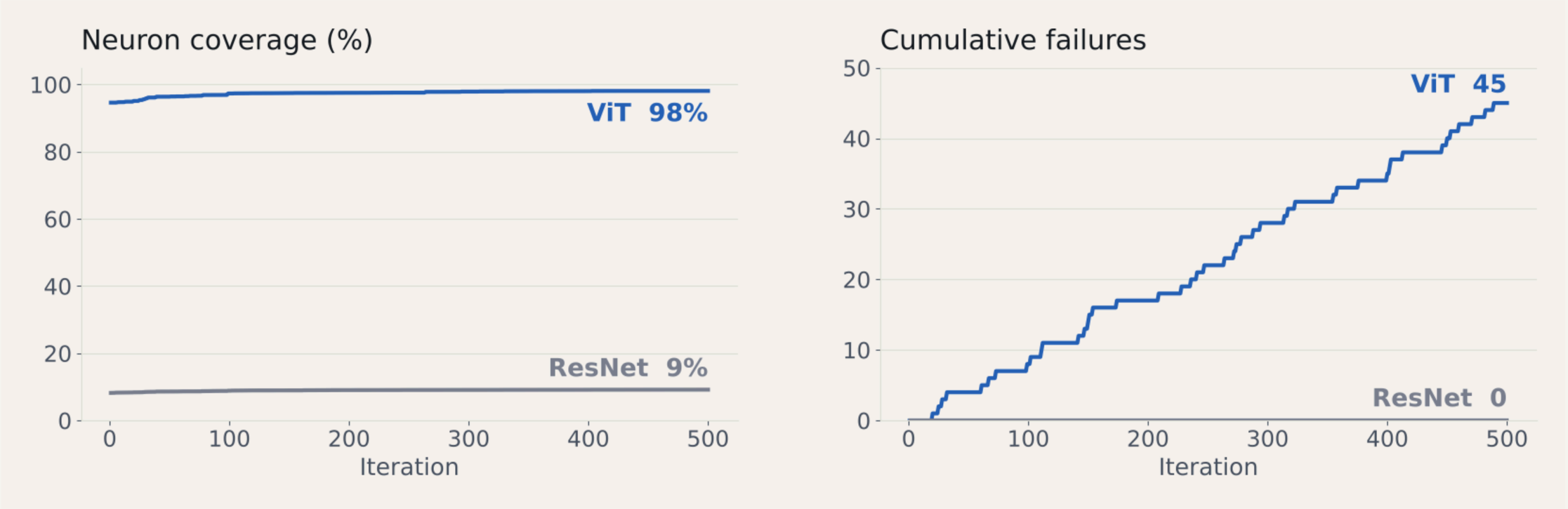
In ViT, brightness caused 38 of the 45 failures



Failure rate = failures ÷ mutants of that type · 500 iterations, ε = 0.08, random weights · AI-generated content · 9

RESULTS · COVERAGE VS. FAILURES

More coverage did not yield more failures



ViT coverage started at 94.6% and ResNet coverage stalled at 9%. ViT failures grew at a steady rate, independent of coverage ($r = -0.31$).

RESULTS · TOKENIZER

The pure-Python tokenizer exceeded max_length

Pure Python · max_length=2 · text 'Ƒ'

`[CLS] x [SEP]`

3 tokens: failure, truncation gives up

Fast (Rust) · same input

`[CLS] [SEP]`

2 tokens: correct

Run	Executions	Coverage	Failures
Fast (Rust)	258,995	141	0
Pure Python	~470	295	**1, in < 1 s**

CONCLUSION

Answers to the research questions

Question	Answer
Do small perturbations change the class?	Yes for ViT: 9% of mutants, with $\varepsilon = 0.08$
Which mutation fails most often?	Brightness: 42% failure rate in ViT
Is ResNet or ViT more robust?	Inconclusive: random weights
Does more coverage find more failures?	No: $r = -0.31$
Does the tokenizer handle any text?	The fast one did; the Python one exceeds max_length

Next step: rerun with the pretrained Hugging Face weights and natural photographs.